

Entregable 4 — Seguridad y Resiliencia

Minimización de datos, rutas de error y puntos de Human-in-the-Loop — Proyecto ADC Triage

1. Minimización de datos

El sistema toca datos personales reales (email, nombre implícito en la firma, contenido del mensaje del cliente), así que se diseñó para tocar lo mínimo indispensable:

Dato	Dónde vive	Por qué se minimizó así
Contenido del email (remitente, asunto, mensaje)	Solo en la base de Airtable del proyecto (memoria propia del sistema) y en el prompt puntual que se le manda a Groq para clasificar/redactar.	No se replica en ningún otro sistema ni se guarda log adicional fuera de Airtable.
Directrices privadas de ADC	Base de Conocimiento RAG en Airtable, consultada solo por categoría relevante.	El agente redactor tiene instrucción explícita de usar EXCLUSIVAMENTE ese contexto — nunca conocimiento externo de internet, así no se filtra ni se inventa información sobre el negocio.
Credenciales (Airtable, Groq, Gmail, Slack)	Credential Manager nativo de n8n.	Ninguna API key vive hardcodeada en el JSON del workflow — se referencian por ID de credencial, reemplazable sin tocar la lógica.
Mensaje que ve el revisor humano en Slack	Solo asunto, remitente, categoría/prioridad y el borrador de respuesta — no se reenvía el hilo completo de Gmail ni metadata innecesaria.	El revisor ve lo mínimo necesario para aprobar con criterio, sin exponer de más en un canal de equipo.

2. Rutas de error (resiliencia ante fallos de API o datos incompletos)

En la primera versión del flujo, el manejo de errores solo cubría los dos nodos de IA (clasificar y redactar). Al revisar el proyecto contra la rúbrica se detectó que un fallo de Airtable, Gmail o Slack —no solo de la IA— podía cortar la ejecución sin dejar rastro. Se corrigió agregando **onError: continueErrorOutput** a los 8 nodos operativos restantes, todos derivando al mismo circuito de registro:

Nodo protegido	Qué falla puede capturar
Clasificar Consulta (IA)	Timeout o error de la API de Groq al clasificar.
Redactar Respuesta (IA)	Timeout o error de la API de Groq al redactar.
Marcar Email como Leído (Gmail)	Fallo de la API de Gmail.
Buscar Base de Conocimiento RAG (Airtable)	Fallo de Airtable al leer las directrices.
Guardar Respuesta Generada (Airtable)	Fallo de Airtable al guardar el borrador.
Notificar en Slack y Esperar Aprobación (HITL)	Fallo de la API de Slack al notificar o esperar.
Marcar Aprobado por Humano (Airtable)	Fallo de Airtable al registrar la aprobación.

Nodo protegido	Qué falla puede capturar
Marcar Rechazado por Humano (Airtable)	Fallo de Airtable al registrar el rechazo.
Enviar Respuesta por Gmail	Fallo de la API de Gmail al enviar la respuesta final.
Marcar como Enviado (Airtable)	Fallo de Airtable al cerrar el ciclo de la consulta.

Cada una de estas salidas de error termina en el mismo patrón de dos pasos: se crea un registro en **Log de Errores** (con tipo de error, detalle crudo del mensaje de la API, timestamp, y link a la consulta que lo originó) y se actualiza la fila correspondiente en **Consultas** a Estado = *Error*. Así ninguna falla queda muda: siempre hay una fila trazable que alguien puede revisar y reprocesar a mano.

Nota honesta: el nodo “**Crear Fila en Airtable (Pendiente)**” — el primer registro que se crea en Consultas apenas llega el email — no tiene todavía onError configurado. Si Airtable rechazara esa escritura puntual (por ejemplo, un campo obligatorio vacío), la ejecución cortaría sin dejar rastro en ese punto específico. Es una mejora pendiente de bajo esfuerzo (agregar el mismo patrón de error que ya tienen los otros 10 nodos), que no se aplicó todavía porque el flujo ya está importado y en pruebas en n8n.

3. Puntos de Human-in-the-Loop (HITL)

Hay un único punto de aprobación humana, pero es el que importa: **antes de contactar al cliente final**. Una vez que la IA redacta la respuesta, el flujo se detiene en un nodo Slack “Send & Wait” (aprobación doble: “Aprobar y Enviar” / “Rechazar”, con timeout de 24 horas) y no continúa hasta que una persona decide. Si aprueba, recién ahí se dispara el envío real por Gmail. Si rechaza, la consulta queda marcada como Descartada con la nota del motivo, sin que el cliente reciba nada generado por IA sin revisar. Esto evita el “Efecto Metralleta” que la consigna del curso pide prevenir explícitamente.

4. Check de seguridad de la consigna

Pregunta de la consigna	Respuesta
¿Tu flujo tiene un filtro para evitar bucles infinitos?	Sí. Doble capa: (1) el email se marca como leído apenas se crea el registro en Airtable, antes de que la IA lo procese, así el próximo polling no lo vuelve a levantar; (2) además se busca por Message ID en Airtable antes de crear cualquier fila nueva, para descartar duplicados aunque el marcado como leído fallara.
¿Estás comparando tipos de datos correctos en tus filtros (ej. número vs número)?	Sí. El router de aprobación humana compara booleano vs booleano (<code>\$json.data.approved === true</code>), y los routers de categoría comparan string vs string. No hay comparaciones de tipos mezclados en ningún IF/Switch del flujo.
¿Tu prompt de IA es dinámico y usa variables del sistema?	Sí. Ambos prompts arman el mensaje del usuario con expresiones dinámicas (<code>{{ \$json... }}</code> / <code>itemMatching</code>) que toman asunto, mensaje y contexto RAG reales de cada ejecución — el system message define el rol y las reglas, pero el contenido siempre viene de la consulta real, nunca hardcoded.